



CMP450: Object-Oriented Software Engineering

Spring 2026

IntelliRide: Heuristic-Based Ride-Sharing & Carpool Matching System

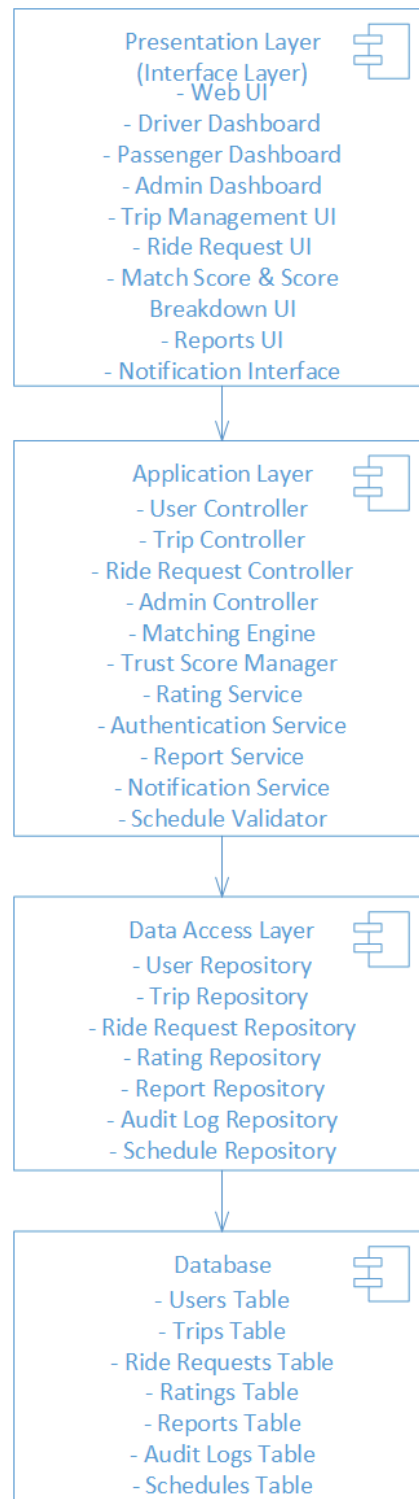
Submitted by:

Name	ID
Mohamed Alawadhi	b00094286
Humaid Almheiri	b00092339
Shaikh Usman	b00092197

Instructor: Dr. Bilal Al-Ahmad

Submission Date: 19th April 2026

1. Layered Architectural Style



The layered architecture of the IntelliRide system is organized into four distinct layers, each with well-defined responsibilities that ensure modularity, scalability, and maintainability.

1. Presentation Layer (User Interface):

This topmost layer provides the general web interface for its users. The users can be a driver, passenger, or even admin. Each having their own specialized dashboard (Driver, Passenger, and Admin Dashboards). Users interact with the system through this layer to request rides, modify trips, view match score breakdowns, view or submit reports, and view notifications.

2. Application Layer:

This layer acts as the controller between the interface and the application. It manages workflows such as users, trips, ride requests, admins. It also handles the matching scores, trust score management, and a schedule validator. While providing services such as rating, authentication, reporting, and notifications.

3. Data Access Layer:

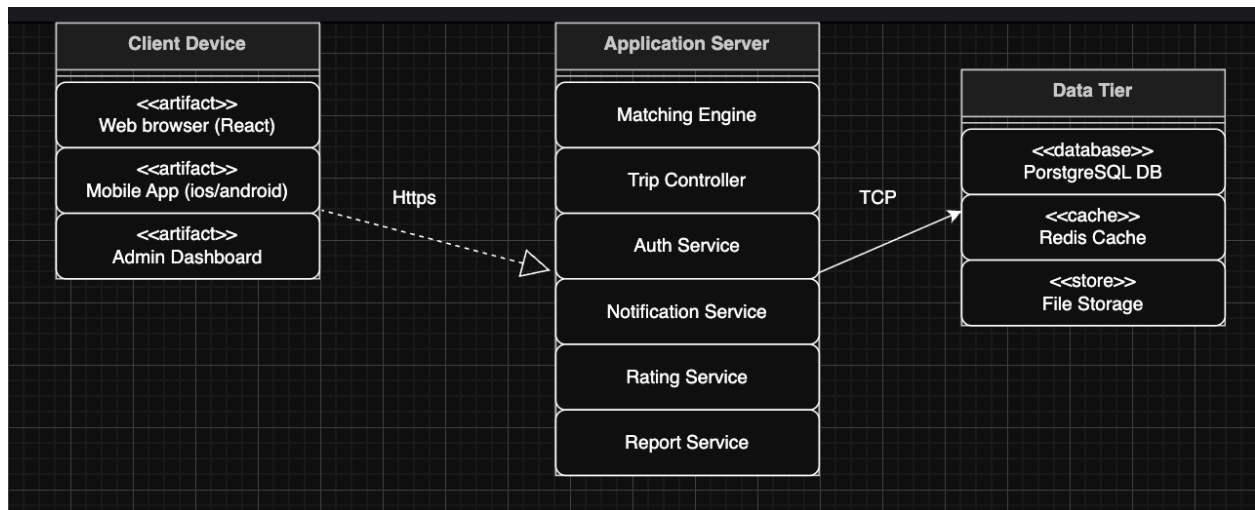
This layer handles all communication between the application and the physical database. It uses repositories such as the User Repository, Trip Repository, Ride Request Repository, Rating Repository, Report Repository, Audit Log Repository, and Schedule Repository to abstract data storage and retrieval, promoting loose coupling and secure data handling.

4. Database Layer:

This foundational layer contains all persistent data in structured tables which includes Users Table, Trips Table, Ride Requests Table, Rating Table, Reports Table, Audit Log Table, and Schedules Table. It ensures data consistency and secure storage of relevant information.

Together, these layers establish a clean, hierarchical design that improves maintainability, supports reusability, and separates presentation from application and data access. This architecture allows each layer to evolve independently, ensuring a flexible and scalable system structure.

2. Client Server Architectural Style Deployment Diagram



2.1 Client Tier

End users interact with IntelliRide through 3 types of client interface: Web browser application built with React, a Mobile App, and an Admin Dashboard. All client interfaces are thin and contain no business logic, they communicate exclusively with the Application Server via HTTP Rest api calls. This will ensure that sensitive data processing remains on server side and clients can be updated without affecting the backend

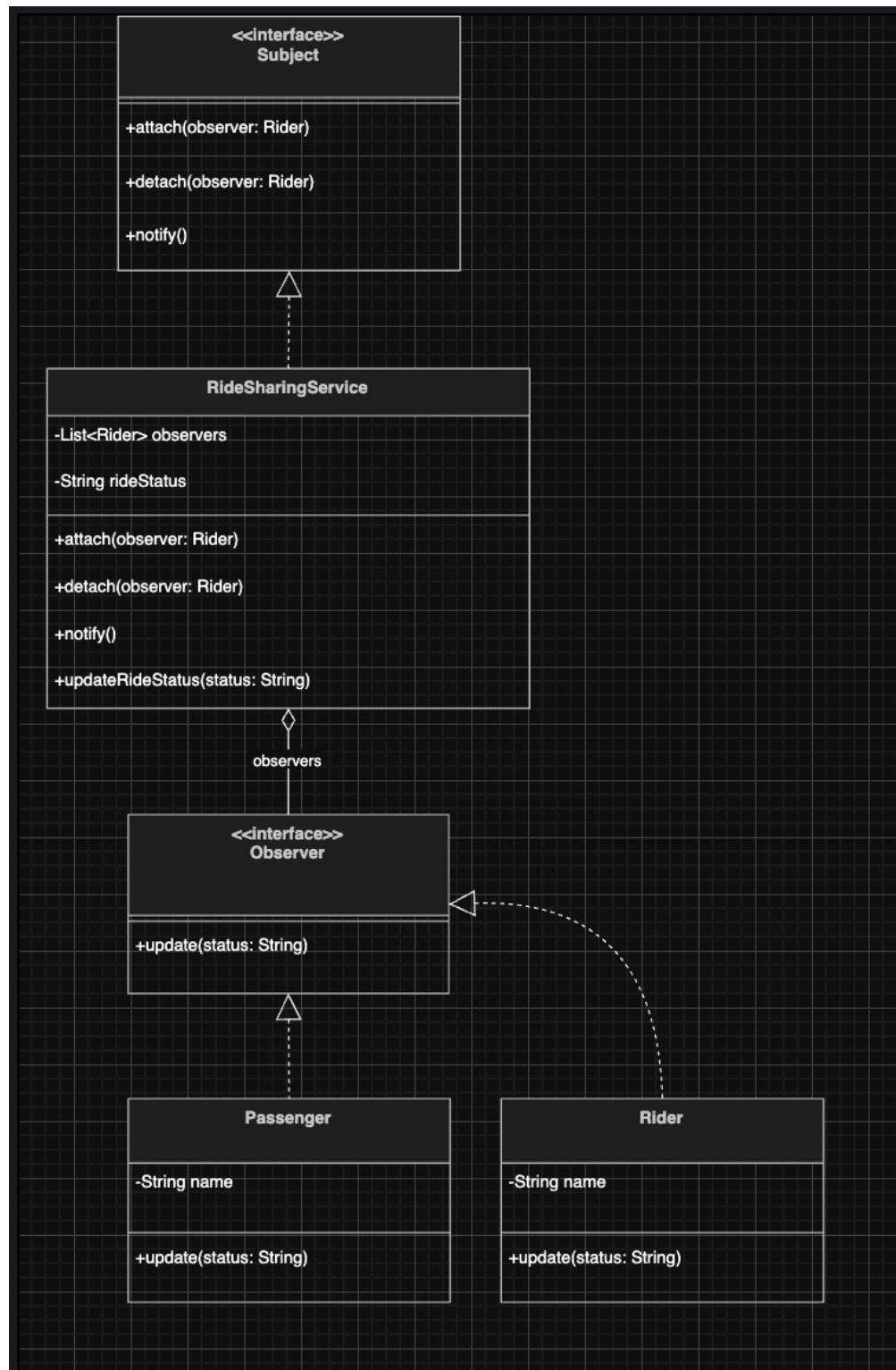
2.2 Application Server Tier

The application Server hosts all business logic components. The Matching engine runs the heuristic scoring algorithm to pair drivers and passengers. The trip controller manages the full ride lifecycle from request to completion. The Auth service handled JWT based authentication and session management. The notification server dispatches push notifications, emails, and sms alerts. The rating service updates trust scores after each completed ride. The report service generates analytics for admins.

2.3 Data Tier

The data tier is isolated from direct client access. It consists for postgresSQL relational db for persistent structured data, a Redis Cache for fast access session and match data, and a file storage service for documents and profile images. The application server is the only tier allowed to communicate with the data tier, enforcing security and preventing unauthorised data access.

3. Design Patterns Observer Pattern



The observer pattern is applied to IntelliRide's real-time notification system. The pattern decouples the ride status change event (the Subject) from the components that need to react to it (the Observers), following the open/closed principle.

Participants:

Subject (interface): Defines the contract for attaching, detaching, and notifying observers.

Methods: attach(observer), detach(observer), notify()

RideSharingService (Concrete Subject): Maintained a List<Rider> and calls notify() whenever updateRideStatus() is invoked, iterating through all registered observers.

Observer (interface): Defines the update (status: String) contract that all concrete observers must implement.

Passenger (Concrete Observer): Receives status updates and refreshes the passenger dashboard in real time.

Rider (Concrete Observer): Receives status updates and triggers push notifications to driver's mobile device.

Why observer fits IntelliRide: Ride status changes (Requested, Matches, In-Progress, Completed, Cancelled) must be communicated instantly to multiple parties, without the RideSharingService needing to know the concrete types of its subscribers. New observer types (ex AdminObserver for audit logging) can be added without modifying existing code.